

Classes: A Deeper Look

Const

composition

static

new

delete

OBJECTIVES

1. To specify const (constant) objects and const member functions.
2. Composition: objects as member of classes
3. To use the this pointer.
4. To use static data members and member functions.
5. To create and destroy objects dynamically with operators new and delete, respectively.

Constant Objects and Constant Member Function

- `const` member functions may be invoked for `const` and non-`const` objects.
- non-`const` member functions can only be invoked for non-`const` objects.
- If a non-`const` member function is invoked on a `const` object, it is a compiler error.
- `const` objects may only be used for `const` parameters.
- `const` methods cannot invoke non-`const` methods.

“The compiler checks that the implementation of a const member function does not alter the data members *directly* or *indirectly*.”

"Directly" means changes a private data member in the body of the code.

"Indirectly" means call a non-const member function or pass the const object as a non-const parameter.

```
#include<iostream>
using namespace std;
```

```
class classone{
private:
    int a;
public:
    classone(int x)
    {
        a = x;
    }
    void geta()
    {
        cout << "Enter a:" << endl;
        cin >> a;
    }
    void showa()
    {
        cout <<"Value of a:="<< a<<endl;
    }
}
```

```
};
```

```
int main()
{
    classone c1(5);
    c1.geta();
    c1.showa();
    const classone c2(25);
    c2.geta();
    c2.showa();
    cin.ignore();
    cin.clear();
    system("pause");
    return 0;
}
```

```
#include<iostream>
using namespace std;
class classone{
private:
    int a;
public:
    classone(int x)
    {
        a = x;
    }
    void geta()
    {
        cout << "Enter a:" << endl;
        cin >> a;
    }
    void showa()const
    {
        cout <<"Value of a:="<< a<<endl;
    }
};

int main()
{
    classone c1(5);
    c1.geta();
    c1.showa();
    const classone c2(25);

    c2.showa();
    cin.ignore();
    cin.clear();
    system("pause");
    return 0;
}
```

Composition: Objects as Members of Classes

1. Sometimes referred to as a *has-a* relationship
2. A class can have objects of other classes as members

```
class Band
{
    Guitarist gtr;
    Drummer drm;
    Vocalist voc;
    ...
};
```

```
class Guitarist
{...};
```

```
class Drummer
{...};
```

```
class Drummer
{...};
```

```
= #include<iostream>
#include<string>
using namespace std;

= class Birthday
{
private:
    int month;
    int date;
    int year;
public:
    Birthday(int m, int d, int y);
    void print();
};

class human{
private:
    string name;
    Birthday dob;
public:
    human(string m, Birthday b);
    void printvalue();

};

Birthday::Birthday(int m , int d, int y)
{
    month = m;
    date = d;
    year = y;

}
```

```
void Birthday::print()
{
    cout << month << "/" << date << "/" << year << endl;
}

human::human(string m, Birthday b) :name(m), dob(b)
{
}

void human::printvalue()
{
    cout << name << endl;
    dob.print();
}

int main()
{
    Birthday b1(01, 01, 1995);
    human h1("sara",b1);
    h1.printvalue();
    cin.ignore();
    cin.clear();
    return 0;
}
```

Observations

If a class member is an object of another class, making that member object public does not violate the encapsulation and hiding of that member object's private members.

However, it does violate the encapsulation and hiding of the containing class's implementation.

So member objects of class types should still be private, like all other data members.

Using the **this** Pointer

1. Member functions know which object's data members to manipulate
 - a. Every object has access to its own address through a pointer called **this** (a C++ keyword)
 - b. An object's this pointer is not part of the object itself
 - c. The this pointer is passed (by the compiler) as an implicit
2. Objects use the **this** pointer implicitly or explicitly
 - a. Implicitly when accessing members directly
 - b. Explicitly when using keyword **this**
 - c. **this** pointer can be used only with member function
 - a. Friend function can't use **this** pointer

static Class Members

1. Only one copy of a variable shared by all objects of a class
 - a. “Class-wide” information
 - b. A property of the class shared by all instances, not a property of a specific object of the class
2. Declaration begins with keyword static
3. Visible only within the class, but its lifetime is the entire program.
4. It is initialized to zero when the first object is created

```
#include<iostream>
#include<string>
using namespace std;

class classone{
private:
    int a;
    static int count;
public:
    void getdata(int x)
    {
        a = x;
        count++;
    }
    void getcount(void)
    {
        cout << "count" << endl;
        cout << count << endl;
    }
};
```

```
int classone::count;
int main()
{
    classone c1, c2, c3;
    c1.getdata(5);
    c2.getdata(6);
    c1.getcount();

    cin.ignore();
    cin.clear();
    return 0;
}
```

Static member function

1. A member function independent of any particular object of the class . It can be called even if no objects of the class exist
2. Accessed using only the class name and the scope resolution operator ::
3. Can only access static data member, other static member functions.
4. A static member function does not have a this pointer
5. When a static member function is called, there might not be any objects of its class in memory

```
#include<iostream>
using namespace std;

class classone{
private:
    int a;
    static int count;
public:
    void getdata(int x)
    {
        a = x;
        cout <<"a="<< a<<endl;
        count++;
    }
    static void getcount()
    {
        cout << "count:=" << endl;
        cout << count << endl;
    }
};
```

```
int classone::count;
int main()
{
    classone c1, c2,c3;
    c1.getdata(5);
    c2.getdata(6);
    classone::getcount();
    cin.ignore();
    cin.clear();
    return 0;
}
```

Common Programming Error

- **Using the this pointer in a static member function is a compilation error.**
- **Declaring a static member function const is a compilation error.**
- **The const qualifier indicates that a function cannot modify the contents of the object in which it operates, but static member functions exist and operate independently of any objects of the class.**

Dynamic Memory Management with Operators new and delete

1. Enables programmers to allocate and de-allocate memory for any built-in or user-defined type.
2. Performed by operators new and delete.

For example,
dynamically allocating memory for an array instead of
using a fixed-size array.

- **Operator new**

1. Allocates (i.e., reserves) storage of the proper size for an object at execution time.
2. Calls a constructor to initialize the object.
3. Returns a pointer of the type specified to the right of new .
4. Can be used to dynamically allocate any fundamental type (such as int or double) or any class type.

- **Free store**

1. Sometimes called the heap.
2. Release of memory assigned to each program for storing objects created at execution time.

- **Operator delete**

1. Destroys a dynamically allocated object
2. Calls the destructor for the object
3. De-allocates (i.e., releases) memory from the free store
4. The memory can then be reused by the system to allocate other objects

Initializing an object allocated by new

1. **Initializer for a newly created fundamental-type variable**

Example

```
double *ptr = new double( 3.14159 );
```

2. **Specify a comma-separated list of arguments to the constructor of an object**

Example

```
Time *timePtr = new Time( 12, 45, 0 );
```

Common Programming Error

- Not releasing dynamically allocated memory when it is no longer needed can cause the system to run out of memory prematurely.
- This is sometimes called a “memory leak.”

new operator can be used to allocate arrays dynamically

Dynamically allocate a 10-element integer array:

```
int *gradesArray = new int[ 10 ];
```

size can be specified using any non-negative integral expression

Delete a dynamically allocated array

```
delete [] gradesArray;
```

1. This de-allocates the array to which gradesArray points
2. If the pointer points to an array of objects
 1. First calls the destructor for every object in the array
 2. Then de-allocates the memory
3. If the statement did not include the square brackets ([]) and gradesArray pointed to an array of objects
 1. Only the first object in the array would have a destructor call

- `#include<iostream>`
- `using namespace std;`
- `class One`
- `{`
- `private:`
- `int *secret;`
- `public:`
- `One()`
- `{`
- `secret = new int;`
- `*secret = 5;`
- `}`
- `One(int x)`
- `{`
- `secret = new int;`
- `*secret = x;`
- `}`
- `void input()`
- `{`
- `cin >> *secret;`
- `}`
- `void Print() const`
- `{`
- `cout << "secret = " << *secret << endl;`
- `}`
- `};`

Common Programming Error

- Using `delete` instead of `delete []` for arrays of objects can lead to runtime logic errors.
- To ensure that every object in the array receives a destructor call, always delete memory allocated as an array with operator `delete []`.
- Similarly, always delete memory allocated as an individual element with operator `delete`.